

Set-UID Programs

该程序不能访问这个x文件，因为该程序运行时的有效用户id是Bob的ID，而Bob没有x的访问权限

该进程不能读取文件，因为该进程的有效用户id = 1000，而文件的读取权限需要用户id=2000

Set-UID程序内部的逻辑设置了该程序能完成的功能，因此恶意用户利用该程序是不能做超出范围的功能的事情的

不行，调试器发现调试的是特权程序后，是不会进行程序的单步调试修改的

因为system的工作方式需要启动shell来运行命令，如果不对system函数调用的参数做严格检查，会导致数据和程序混在一起，从而引起安全问题。另外，从攻击面上来说多了一个shell的攻击点，所以不安全

Copyright © 2017 by Wenliang Du, All rights reserved.

Personal uses are granted. Use of these problems in a class is granted only if the author's book is adopted as a textbook of the class. All other uses must seek consent from the author.

S2.1. Alice runs a Set-UID program that is owned by Bob. The program tries to read from /tmp/x, which is readable to Alice, but not to anybody else. Can this program successfully read from the file?

S2.2. A process tries to open a file for read. The process's effective user ID is 1000, and real user ID is 2000. The file is readable to user ID 2000, but not to user ID 1000. Can this process successfully open the file?

S2.3. A root-owned Set-UID program allows a normal user to gain the root privilege while executing the program. What prevents the user from doing bad things using the privilege?

S2.4. We are trying to turn a program prog owned by the seed user into a Set-UID program that is owned by root. Can running the following commands achieve the goal?

```
$ sudo chmod 4755 prog
$ sudo chown root prog
```

不行，因为 chown 命令会对 EUID 复位，使之与 RUID xia

S2.5. The chown command automatically disables the Set-UID bit, when it changes the owner of a Set-UID program. Please explain why it does that.

S2.6. When we debug a program, we can change the program's internal variables during the execution. This can change a program's behavior. Can we use this technique to debug a Set-UID program and change its behavior? For example, if the program is supposed to open the /tmp/xyz file, we can modify the filename string, so the Set-UID program ends up opening /etc/passwd.

S2.7. Both system() and execve() can be used to execute external programs. Why is system() unsafe while execve() is safe?

S2.8. When a program takes an input from users, we can redirect the input device, so the program can take the input from a file. For example, we can use prog < myfile to provide the data from myfile as input to the prog program. Now, if prog is a root-owned Set-UID program, can we use the following method to get this privileged program to read from the /etc/shadow file?

```
$ prog < /etc/shadow
```

S2.9. When a parent Set-UID process (effective user ID is root, and the real user ID is bob) creates a child process using fork(), the standard input, output, and error devices of the parent will be inherited by the child. If the child process drops its root privilege, it still retains the access right to these devices. This seems to be a capability leaking, similar to the capability-leaking case covered in this chapter. Can this pose any danger?

这要看在有特权的时间段，这三个文件描述符是否关联，重定向到了需要特权才能够访问的文件，如果回答是yes，那么降低权限后，就有权限泄漏的风险，如果回答是no，那么没有权限泄漏风险。fd0,1,2的默认值分别是终端输入/dev/tty (read)，屏幕/dev/tty (write)、屏幕/dev/tty (write)

S2.10. ★

The superuser wants to give Alice a permission to view all the files in the system using the more command. He does the following:

```
$ cp /bin/more /tmp/mymore
$ sudo chown root /tmp/mymore
$ sudo chmod 4700 /tmp/mymore
```

Basically, the above commands turns /tmp/mymore into a Set-UID program. Right now, because the permission is set to 4700, other users cannot execute the program. The superuser uses another command (now shown) to grant the execution permission only to Alice. We are not assuming that Alice is completely trusted. It is OK if Alice can only read other people's files, but it is not OK if Alice can gain any privilege beyond that, such as writing to other people's files. Please read the manual of the more program and find out what Alice can do to gain more privilege.

S2.11. Assume that you have a file that you would allow other users to read, only if a user's ID is smaller than 1000. Please describe how you can actually achieve this.

S2.12. Sam found a very useful web page, which contains links to many interesting papers. He wants to download those papers. Instead of clicking on each of the links, he wrote a program that parses a HTML web page, get the papers' URLs from the web page, and then use a program called wget to fetch each identified URL. The following is the code snippet:

```
char command[100];
char* line, url;
line = getNextLine(file); // Read in one line from the HTML file.
while (line != NULL) {
    // Parse the line to get a URL string.
    url = parseURL (line);
    if (url != NULL){
        // construct a command, and execute it
        sprintf(command, "%s %s", "wget", url);
        system(command);
    }
    line = GetNextLine(file);
}
```

The function sprintf() is quite similar to printf(), except that sprintf() puts the output in a buffer pointed by the first argument, while printf() sends the output to the display. Please be noted that the functions getNextLine() and parseURL() are also implemented by Sam (their code is not displayed here). The program wget is a command-line program in Unix that can be used to download web files from a given URL.

The owner of the web page knows what Sam is doing with his page; he wants to attack Sam's program. He knows the code above, but he does not know how Sam implements GetNextLine() or ParseURL(), but he suspects that Sam may make some mistakes there. (1) If you are the attacker, please describe how you plan to attack. (2) How do you fix the problem?

笨办法是一个一个的设置，耗时费力；合理的方法是写一个程序，任一个用户想访问该文件，只能通过这个程序进行，通过程序判断用户的编号是否小于1000

程序第三行会从html文件中读取一行数据，而html是个外部数据，由攻击者控制，那么我们读取的这一行数据就可能带来风险，而且在程序中用system系统调用执行的命令中包含了这一行的数据，攻击者就可以利用这一点，在url中写入一些命令，然后让程序的system来执行；对应的措施是，使用execve这样的api来提高安全性，不经过bash，还有就是限定url字符串的长度，如果超过100，可能导致缓冲区溢出攻击。